Docker Assignment 7

Docker Networking – Default Bridge, Custom Bridge and Container Communication

Objective

Create containers on Docker's default bridge and a user-defined bridge network; inspect their IP addresses; verify same-network connectivity; and demonstrate connecting a container to an additional network.

Network Architecture

Docker Host (EC2 / Linux)
 ├─ Default bridge (bridge): c1, c2 [Docker-assigned subnet]
 └─ Custom bridge (mybridge): v1, v2 [10.10.0.0/16; gateway 10.10.0.1]
Optional: connect c2 to mybridge as a second network interface.

Note: Default bridge containers generally communicate using IP addresses; user-defined bridge containers can resolve one another by container name. Separate bridge networks are isolated by default.

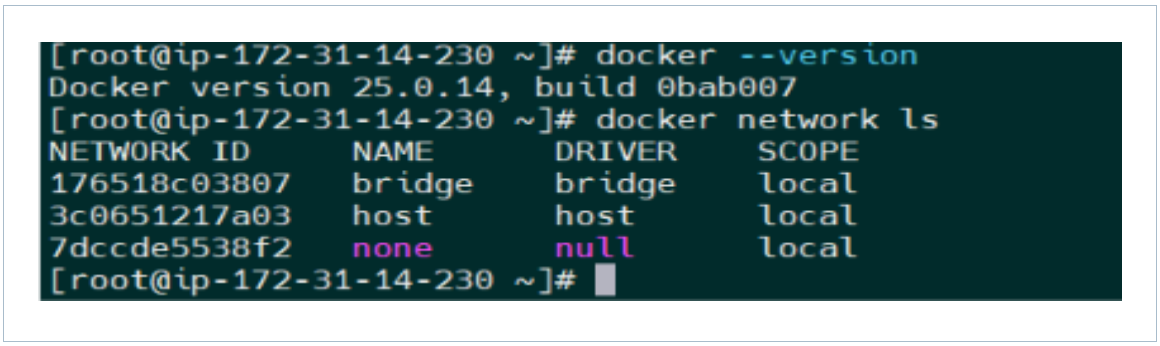
Step 1 – Verify Docker and existing networks

Connect to your EC2 instance and verify Docker is available.

```
docker --version

docker network ls
```

Screenshot: Docker version and network list



Step 2 – Create custom bridge network

Create mybridge with the subnet and gateway from the assignment diagram. Ensure the subnet does not overlap with existing networks.

```
docker network create --driver bridge --subnet 10.10.0.0/16 --gateway 10.10.0.1
mybridge

docker network ls
```

Screenshot: Custom bridge network creation

```
[root@ip-172-31-14-230 ~]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-14-230 ~]# docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
176518c03807   bridge    bridge    local
3c0651217a03   host      host      local
7dccde5538f2   none      null      local
[root@ip-172-31-14-230 ~]# docker network create --driver bridge --subnet 10.10.0.0/16 --gateway 10.10.0.1 mybridge
54b40c5b6c6bdea1403f8ee8af43ad793a1b5e01e3994f04178457de9f9ceac4
[root@ip-172-31-14-230 ~]# docker network ls
NETWORK ID     NAME       DRIVER    SCOPE
176518c03807   bridge     bridge    local
3c0651217a03   host       host      local
54b40c5b6c6b   mybridge   bridge    local
7dccde5538f2   none       null      local
[root@ip-172-31-14-230 ~]#
```

Step 3 – Start containers on default bridge

Start two lightweight Alpine containers on the default bridge network.

```
docker run -dit --name c1 --network bridge alpine sh
```

```
docker run -dit --name c2 --network bridge alpine sh
```

Screenshot: c1 and c2 created

```
[root@ip-172-31-14-230 ~]# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED         STATUS         PORTS     NAMES
bd67cbfcacba   alpine    "sh"      6 minutes ago   Up 6 minutes             c2
100bac920f9c   alpine    "sh"      6 minutes ago   Up 6 minutes             c1
[root@ip-172-31-14-230 ~]# docker network inspect bridge
[
    {
        "Name": "bridge",
        "Id": "176518c03807ffb74e854422ebb784fd072f71f858b460ca216f3f9e907d6d18",
        "Created": "2026-10-16T15:37:06.511667033Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "100bac920f9c71ec0b86470a6b2e0c0af4deebf68fd7a710f704412dcba04464": {
                "Name": "c1",
                "EndpointID": "37a749d6fb3b7777ea1a665e2474418f292362ae6d2a79ae6750ebd57c291541",
                "MacAddress": "02:42:ac:11:00:02",
                "IPv4Address": "172.17.0.2/16",
                "IPv6Address": ""
            },
            "bd67cbfcacbae941a5bd825766105cacc941ed8a641beb39360dcb64debe1054": {
                "Name": "c2",
                "EndpointID": "60521658eb2ef19b25b3ea0078956d6eada8b0f654d4da1e49a76f629232433c",
                "MacAddress": "02:42:ac:11:00:03",
                "IPv4Address": "172.17.0.3/16",
                "IPv6Address": ""
            }
        },
        "Options": {
            "com.docker.network.bridge.default_bridge": "true",
            "com.docker.network.bridge.enable_icc": "true",
            "com.docker.network.bridge.enable_ip_masquerade": "true",
            "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
            "com.docker.network.bridge.name": "docker0",
            "com.docker.network.driver.mtu": "1500"
        },
        "Labels": {}
    }
]
[root@ip-172-31-14-230 ~]#
```

Step 4 – Start containers on custom bridge

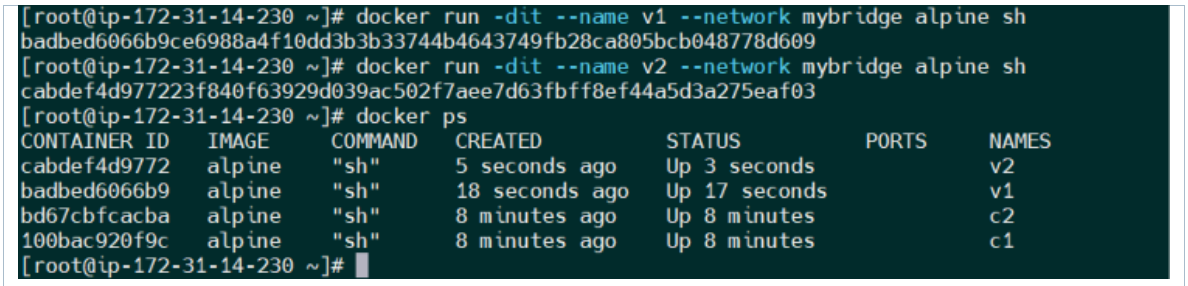
Start two containers on mybridge.

```
docker run -dit --name v1 --network mybridge alpine sh

docker run -dit --name v2 --network mybridge alpine sh

docker ps
```

Screenshot: All four running containers



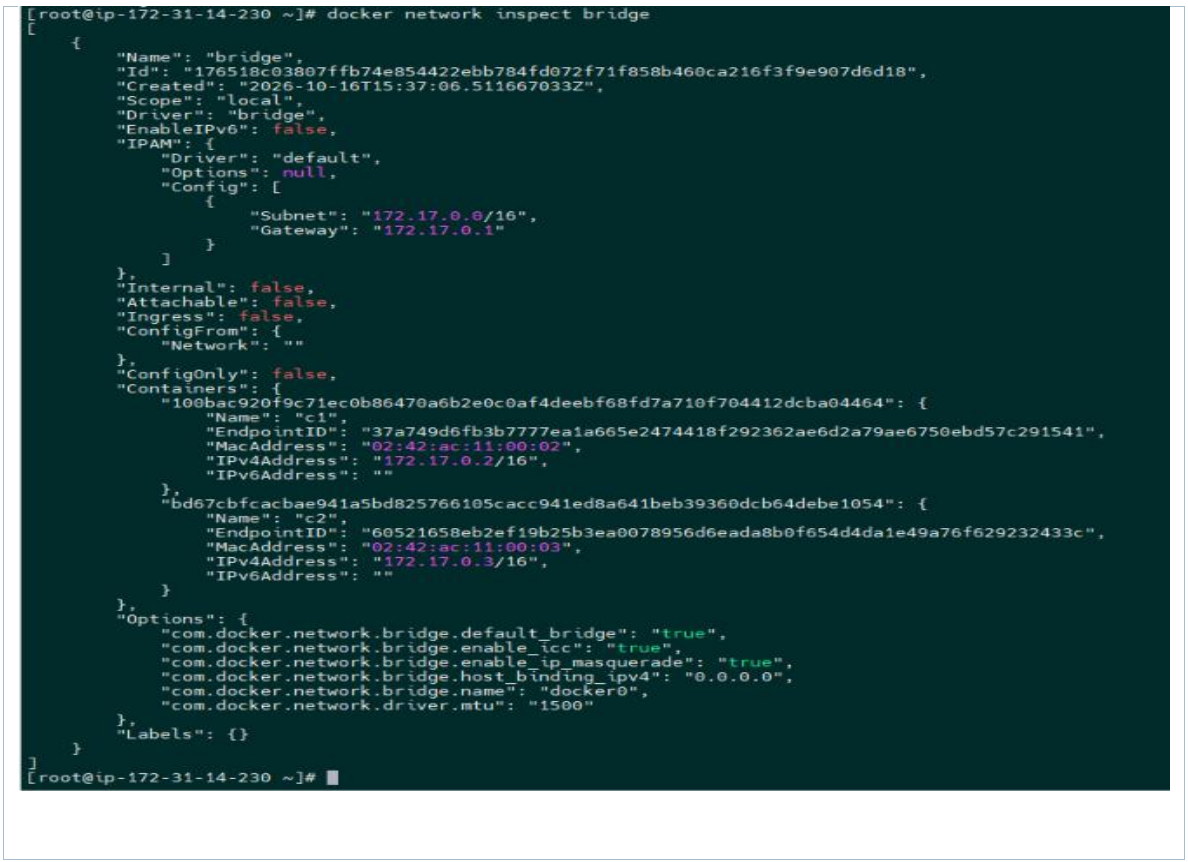
Step 5 – Inspect network IP addresses

Check subnet, gateway and attached containers. Actual default-bridge addresses may differ.

```
docker network inspect bridge

docker network inspect mybridge
```

Screenshot: Network inspect output



```
[root@ip-172-31-14-230 ~]# docker network inspect mybridge
[
    {
        "Name": "mybridge",
        "Id": "54b40c5b6c6bdea1403f8ee8af43ad793a1b5e01e3994f04178457de9f9ceac4",
        "Created": "2026-10-16T16:56:02.779909474Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "10.10.0.0/16",
                    "Gateway": "10.10.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "badbed6066b9ce6988a4f10dd3b3b33744b4643749fb28ca805bcb048778d609": {
                "Name": "v1",
                "EndpointID": "3bf13c47b97de6a43456ad5726c1f73b89302aabeae3b6106a5ba51846de862d",
                "MacAddress": "02:42:0a:0a:00:02",
                "IPv4Address": "10.10.0.2/16",
                "IPv6Address": ""
            },
            "cabdef4d977223f840f63929d039ac502f7aee7d63fbff8ef44a5d3a275eaf03": {
                "Name": "v2",
                "EndpointID": "189e134fc63fb0b79967099de7b981933ab867604ca5f45b2129c58ed99006a5",
                "MacAddress": "02:42:0a:0a:00:03",
                "IPv4Address": "10.10.0.3/16",
                "IPv6Address": ""
            }
        },
        "Options": {},
        "Labels": {}
    }
]
[root@ip-172-31-14-230 ~]#
```

Step 6 – Test same-network connectivity

Install ping in the source containers. v1 can resolve v2 by name on the user-defined bridge. For the default bridge, obtain c2's IP first.

```
docker exec v1 apk add --no-cache iputils

docker exec c1 apk add --no-cache iputils

docker exec v1 ping -c 4 v2

docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' c2

docker exec c1 ping -c 4 <C2_IP_ADDRESS>
```

Screenshot: Successful ping on both networks

```
[root@ip-172-31-14-230 ~]# docker exec v1 apk add --no-cache iputils
OK: 8455 KiB in 22 packages
[root@ip-172-31-14-230 ~]# docker exec v1 ping -c 4 v2
PING v2 (10.10.0.3) 56(84) bytes of data.
64 bytes from v2.mybridge (10.10.0.3): icmp_seq=1 ttl=127 time=0.092 ms
64 bytes from v2.mybridge (10.10.0.3): icmp_seq=2 ttl=127 time=0.063 ms
64 bytes from v2.mybridge (10.10.0.3): icmp_seq=3 ttl=127 time=0.074 ms
64 bytes from v2.mybridge (10.10.0.3): icmp_seq=4 ttl=127 time=35.3 ms

--- v2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3145ms
rtt min/avg/max/mdev = 0.063/8.884/35.307/15.255 ms
[root@ip-172-31-14-230 ~]#
```

```
[root@ip-172-31-14-230 ~]# docker exec c1 apk add --no-cache iputils
(1/6) Installing libcap2 (2.78-r0)
(2/6) Installing iputils-arping (20250605-r2)
(3/6) Installing iputils-clockdiff (20250605-r2)
(4/6) Installing iputils-ping (20250605-r2)
(5/6) Installing iputils-tracepath (20250605-r2)
(6/6) Installing iputils (20250605-r2)
Executing busybox-1.37.0-r31.trigger
OK: 8455 KiB in 22 packages
[root@ip-172-31-14-230 ~]# docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' c2
172.17.0.3
[root@ip-172-31-14-230 ~]# docker exec c1 ping -c 4 172.17.0.3
PING 172.17.0.3 (172.17.0.3) 56(84) bytes of data.
64 bytes from 172.17.0.3: icmp_seq=1 ttl=127 time=0.159 ms
64 bytes from 172.17.0.3: icmp_seq=2 ttl=127 time=0.152 ms
64 bytes from 172.17.0.3: icmp_seq=3 ttl=127 time=0.066 ms
64 bytes from 172.17.0.3: icmp_seq=4 ttl=127 time=0.144 ms

--- 172.17.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3082ms
rtt min/avg/max/mdev = 0.066/0.130/0.159/0.037 ms
[root@ip-172-31-14-230 ~]#
```

Step 7 – Test isolation between bridges

Before connecting networks, a container attached only to mybridge cannot directly reach a container attached only to the default bridge under normal Docker bridge isolation.

```
docker inspect c2
```

```
docker inspect v1
```

Step 8 – Connect one container to both networks

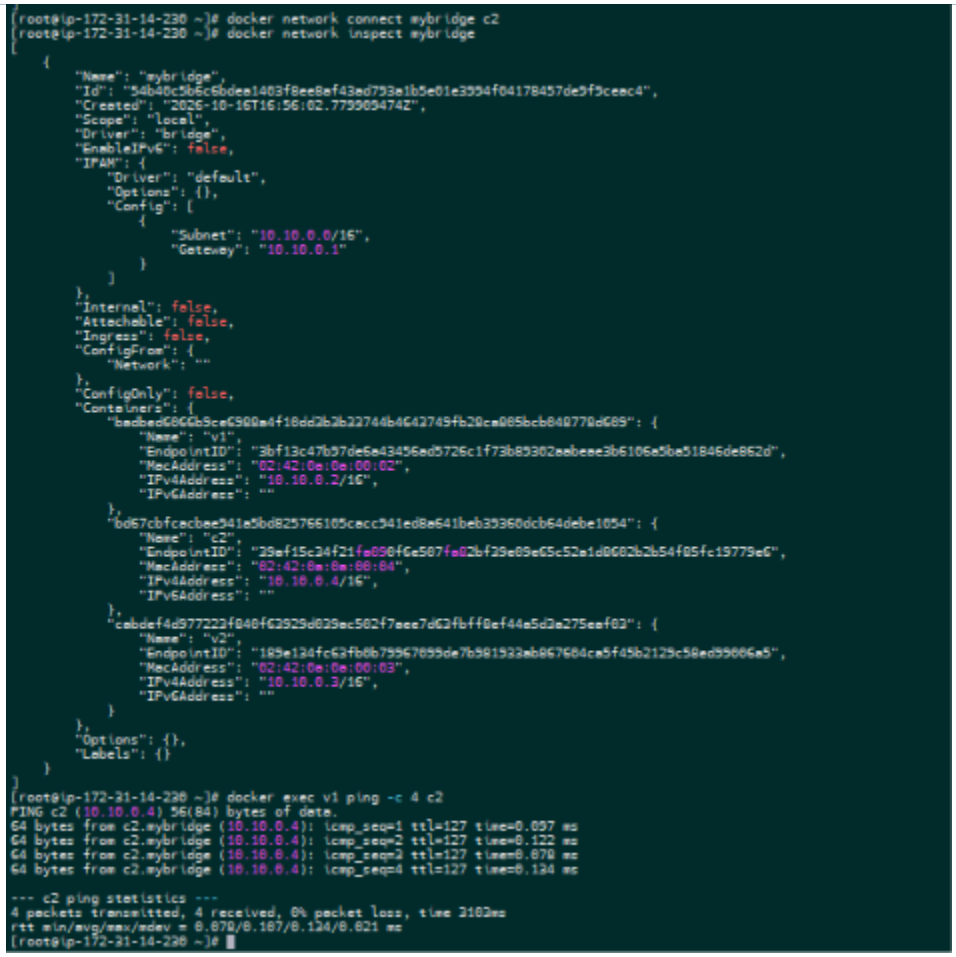
This demonstrates the cross-network arrow in the diagram: c2 becomes a member of both bridge and mybridge. Test from v1 to c2 by name.

```
docker network connect mybridge c2
```

```
docker network inspect mybridge
```

```
docker exec v1 ping -c 4 c2
```

Screenshot: c2 on both networks and successful ping



Expected Results

- Four containers run successfully.
- c1 and c2 share the default bridge; v1 and v2 share mybridge.
- Containers on the same network communicate successfully.
- After connecting c2 to mybridge, v1 can communicate with c2 using its container name.

Optional Cleanup

docker rm -f c1 c2 v1 v2

docker network rm mybridge

Conclusion

Docker bridge networking provides container-to-container connectivity and isolation. User-defined bridges support automatic DNS resolution by container name. A container may be connected to multiple Docker networks when required.